Hops, Store-and-Forward and Pipelining

How to calculate delays across several nodes · based on Basics slides 18, 31, 38, 51–53

The key formula: N packets across k intermediate nodes take $(N + k) \times T_{packet} + hops \times T_{prop}$, where hops = k + 1.

1. What is a hop?

A **hop** is **one link**: one jump from one machine to the next.

```
Src ──hop 1──> R1 ──hop 2──> R2 ──hop 3──> Dst
```

Hops = intermediate nodes + 1. This path has 2 intermediate nodes (R1 and R2), so it has **3 hops**.

> **Exam trap:** questions usually give the number of *nodes*, not hops. “7 intermediate nodes” means **8 hops**.

2. What happens on each hop

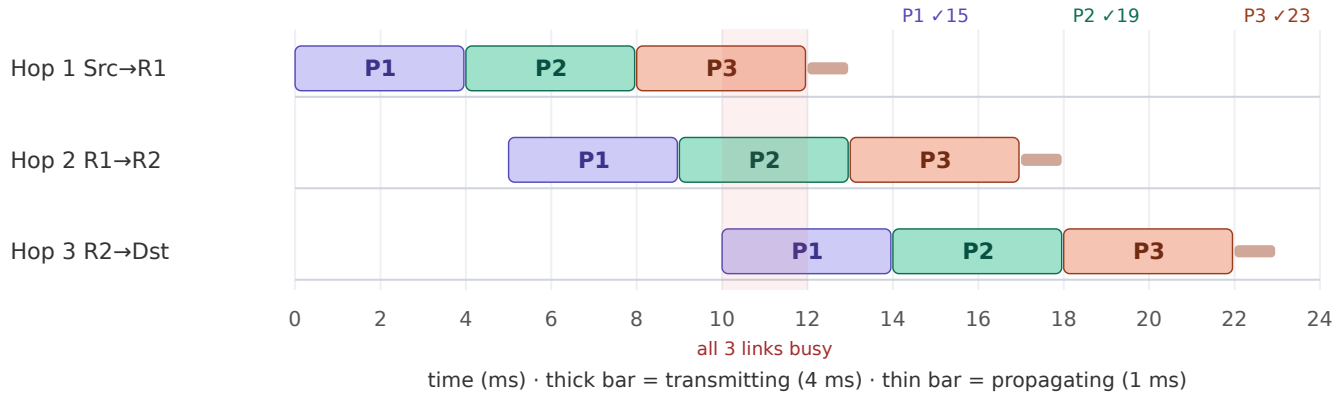
Delay	Formula	Meaning
Transmission	size (bits) ÷ bandwidth (bits/s)	The time to push all the bits onto the wire
Propagation	length (m) ÷ speed (m/s)	The time for one bit to travel along the wire
Queueing	packets waiting × T_{trans}	The time spent waiting behind other packets at a node

Store-and-forward (slide 51): a router must receive the **whole packet** before it starts sending it on. That's why the transmission time is paid again at every hop.

3. Worked example: 3 packets through 2 routers

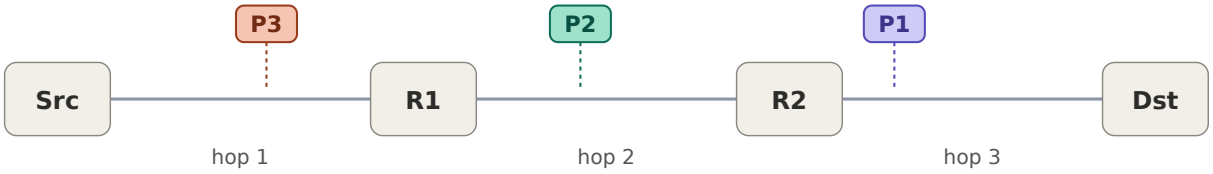
Given	Value
Path	Src → R1 → R2 → Dst (k = 2 nodes, 3 hops)
Packets	N = 3 (P1, P2, P3)
Transmission per packet	T = 4 ms (1,000 B at 2 Mb/s)
Propagation per link	1 ms

Timeline: when each link is busy



Each row is one hop. A packet can only start on the next hop once it has fully arrived (the end of its thin bar). The ticks at the top mark when each packet reaches Dst.

Snapshot at t = 11 ms: every link carries a different packet



*The source is sending P3, R1 is forwarding P2, and R2 is forwarding P1, all at the same time. This is **pipelining**.*

1. **0–4 ms:** the source transmits P1 onto hop 1.
2. **4–5 ms:** P1's last bit propagates to R1. Meanwhile, the source has already started sending P2.
3. **5 ms:** P1 has fully arrived at R1, so R1 can now start forwarding it on hop 2 (store-and-forward).
4. **10–12 ms:** all three links are busy at once. The source is sending P3, R1 is sending P2, and R2 is sending P1.
5. **15 ms:** P1 arrives. **19 ms:** P2 arrives. **23 ms:** P3 arrives, so the whole transfer is done.

4. The formula and the check

Delay = $(N + k) \times T_{packet} + hops \times T_{prop} = (3 + 2) \times 4 + 3 \times 1 = 20 + 3 =$ 23 ms ✓

Why it works: follow the **last** packet, P3.

What P3 waits for	Sum	Time
The source sends P1 and P2, then P3 itself	N × T = 3 × 4	12 ms
R1 and R2 each re-send it	k × T = 2 × 4	8 ms
It propagates along each of the 3 wires	hops × P = 3 × 1	3 ms
Total		**23 ms**

Propagation is counted **once per hop**, not once per packet, because the packets travel along the wires at the same time.

5. Why pipelining matters, and the other cases

If each packet had to cross the *whole* path before the next one started, one packet would take 3 × (4 + 1) = 15 ms, and 3 packets would take **45 ms**. Pipelining cuts that to **23 ms**, almost half (slide 31: transmissions are “pipelined (reducing the overall delay)”).

Case	Formula	Lecture example
One packet	hops × (T_{trans} + T_{prop})	Slide 51: 8 nodes, 1,000 bits, 64 kb/s, no propagation: 9 × 15.625 = **141 ms**
N packets (pipelined)	(N + k) × T_{packet} + hops × T_{prop}	The example above: **23 ms**
Circuit switching	setup + message ÷ rate + hops × T_{prop}	No store-and-forward: once the circuit is set up, the bits stream straight through
With queueing	each node adds (q + 1) × T_{trans}, where q = packets waiting	Slide 53: 5 waiting at each of 8 nodes: 15.625 + 8 × 6 × 15.625 ≈ **766 ms**

> **Beyond the notes:** the general (N + k) form isn't written on your slides, but it follows from the store-and-forward example on slide 51 (N = 1 gives (1 + 8) × 15.625 = 141 ms).

6. Practice question

A 1,500-byte packet crosses **3 intermediate nodes**. Each link runs at 12 Mb/s with 2 ms propagation.
(a) What is the total delay for **one** packet? (b) What is it for **10** packets?

Worked solution

Step 1: write down what you're given.

Quantity	Value
Packet size	1,500 bytes
Intermediate nodes (k)	3
Bandwidth per link	12 Mb/s = 12 × 10^6 bits/s
Propagation per link	2 ms

Step 2: count the hops.

```
Src ──hop 1──> N1 ──hop 2──> N2 ──hop 3──> N3 ──hop 4──> Dst
```

Hops = k + 1 = 3 + 1 = **4 hops**.

Step 3: work out the transmission time for one packet.

1. Convert bytes to bits: 1,500 × 8 = **12,000 bits**.
2. Divide by the bandwidth: 12,000 ÷ 12,000,000 = 0.001 s = **1 ms**.

So T_{trans} = 1 ms and T_{prop} = 2 ms.

(a) One packet

With one packet there is nothing to overlap. On every hop it is transmitted (store-and-forward) and then propagates.

Hop	Transmission	Propagation	Arrives at
1 (Src → N1)	0–1 ms	1–3 ms	N1 at 3 ms
2 (N1 → N2)	3–4 ms	4–6 ms	N2 at 6 ms
3 (N2 → N3)	6–7 ms	7–9 ms	N3 at 9 ms
4 (N3 → Dst)	9–10 ms	10–12 ms	Dst at **12 ms**

Delay = hops × (T_{trans} + T_{prop}) = 4 × (1 + 2) = 12 ms

(b) Ten packets

Now the packets pipeline: while N1 forwards packet 1, the source is already sending packet 2. Use the formula with N = 10 and k = 3:

Delay = (N + k) × T_{packet} + hops × T_{prop} = (10 + 3) × 1 + 4 × 2 = 13 + 8 = 21 ms

Check it by following the last packet (P10):

What P10 waits for	Sum	Time
The source sends P1 to P9, then P10 itself	N × T = 10 × 1	10 ms
N1, N2 and N3 each re-send it (store-and-forward)	k × T = 3 × 1	3 ms
It propagates along each of the 4 links	hops × P = 4 × 2	8 ms
Total		**21 ms**

The same thing timed hop by hop for P10: it leaves the source at 9–10 ms, reaches N1 at 12 ms, is forwarded 12–13 ms and reaches N2 at 15 ms, is forwarded 15–16 ms and reaches N3 at 18 ms, and is forwarded 18–19 ms and reaches Dst at **21 ms** ✓. It never has to wait at a node, because the packet ahead of it (P9) always finishes on each link exactly as P10 arrives.

> **Compare:** sent one at a time with no pipelining, 10 packets would take 10 × 12 = **120 ms**. Pipelining brings it down to **21 ms**.

Answers: (a) **12 ms** (b) **21 ms**